Day – 6 Pandas Library

What is Pandas?

Pandas is a Python library used for working with data sets.It has functions for analyzing, cleaning, exploring, and manipulating data.The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Why Use Pandas?

Pandas allows us to analyze big data and make conclusions based on statistical theories.Pandas can clean messy data sets, and make them readable and relevant.Relevant data is very important in data science.

Pandas Data Structures

Series:

One Dimensional labeled array capable of holiding any data type (integers,floats,string etc).

It similar to a one dimensional NumPy Array but comes with additional functionalities like custom indexing.

```
import pandas as pd
a = [1, 7, 2]
myvar = pd.Series(a)
print(myvar)
```

```
0    1
1    7
2    2
dtype: int64
```

DataFrame:

A two dimensional,size-mutable, heterogeneous tabular data structure with labeld axes (rows and columns). Data sets in Pandas are usually multi-dimensional tables, called DataFrames.

Series is like a column, a DataFrame is the whole table.

```
import pandas as pd
data = {
  "calories": [420, 380, 390],
  "duration": [50, 40, 45]
}
myvar = pd.DataFrame(data)
print(myvar)
```

```
   calories  duration
0       420        50
1       380        40
2       390        45
```

Read CSV Files

A simple way to store big data sets is to use CSV files (comma separated files).CSV files contains plain text and is a well know format that can be read by everyone including Pandas.

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.to_string())
```

	Duration	Pulse	Maxpulse	Calories
0	60	110	130	409.1
1	60	117	145	479.0
2	60	103	135	340.0
3	45	109	175	282.4
4	45	117	148	406.0
5	60	102	127	300.5
6	60	110	136	374.0
7	45	104	134	253.3
8	30	109	133	195.1
9	60	98	124	269.0
10	60	103	147	329.3

Importing the data from a CSV file to a Pandas Dataframe

```
housing_df=pd.read_csv('/content/sample_data/california_housing_train.csv')
type(housing_df)
housing_df.shape
```

```
(17000, 9)
```

Operations Performed:

```
#Finding the number of rows and columns
print("The number of rows and columns are:",housing_df.shape)
#First 5 rows
housing_df.head()
#Last 5 rows
housing_df.tail()
#random 10 rows
housing_df.sample(10)
```

```
The number of rows and columns are: (17000, 9)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
9645	-119.75	36.80	25.0	2716.0	504.0	1257.0	465.0	2.3333	90600.0
16595	-122.70	38.23	47.0	2090.0	387.0	1053.0	377.0	3.5673	310300.0
15248	-122.27	37.53	43.0	1145.0	230.0	586.0	254.0	3.5000	267400.0
7960	-118.41	34.07	52.0	3562.0	764.0	1183.0	361.0	15.0001	500001.0
5192	-118.13	33.86	37.0	2259.0	425.0	1183.0	413.0	5.1805	201600.0

```
#information about the dataframe
housing_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           17000 non-null  float64
 1   latitude            17000 non-null  float64
 2   housing_median_age  17000 non-null  float64
 3   total_rooms         17000 non-null  float64
 4   total_bedrooms      17000 non-null  float64
 5   population          17000 non-null  float64
 6   households          17000 non-null  float64
 7   median_income       17000 non-null  float64
 8   median_house_value  17000 non-null  float64
dtypes: float64(9)
memory usage: 1.2 MB
```

```
housing_df.isnull().sum()
```

	0
longitude	0
latitude	0
housing_median_age	0
total_rooms	0
total_bedrooms	0
population	0
households	0
median_income	0
median_house_value	0

dtype: int64

```
#counting the values based on the labels
housing_df.value_counts('housing_median_age')
```

housing_median_age	count
52.0	1052
36.0	715
35.0	692
16.0	635
17.0	576
34.0	567
33.0	513
26.0	503

```
#group the values based on the mean
housing_df.groupby('housing_median_age').mean()
```

housing_median_age	longitude	latitude	total_rooms	total_bedrooms	population	households	median_income	median_house_value
1.0	-121.465000	37.940000	2158.000000	335.500000	637.000000	190.000000	4.756800	190250.000000
2.0	-119.036306	35.410816	5237.102041	871.448980	2005.224490	707.122449	5.074237	229438.836735
3.0	-118.798478	35.184783	6920.326087	1190.826087	2934.673913	1030.413043	5.572013	239450.043478
4.0	-118.805093	34.987764	6065.614907	1068.192547	2739.956522	964.291925	5.196056	230054.105590
5.0	-118.789497	35.095327	4826.261307	910.924623	2456.979899	826.768844	4.732460	211035.708543
6.0	-118.915969	35.183798	4365.062016	821.550388	2111.263566	746.744186	4.463901	206768.240310
7.0	-119.165497	35.027616	5672.907285	1047.360993	2705.483444	944.112583	4.358441	188445.059603

```
#count or number of values
housing_df.count()
```

	0
longitude	17000
latitude	17000
housing_median_age	17000
total_rooms	17000
total_bedrooms	17000
population	17000
households	17000
median_income	17000
median_house_value	17000

dtype: int64

```
#mean value-column wise
housing_df.mean()
```

	0
longitude	-119.562108
latitude	35.625225
housing_median_age	28.589353
total_rooms	2643.664412
total_bedrooms	539.410824
population	1429.573941
households	501.221941
median_income	3.883578
median_house_value	207300.912353

dtype: float64

```
#standard deviation
housing_df.std()
```

	0
longitude	2.005166
latitude	2.137340
housing_median_age	12.586937
total_rooms	2179.947071
total_bedrooms	421.499452
population	1147.852959
households	384.520841
median_income	1.908157
median_house_value	115983.764387

dtype: float64

```
#minimum value
housing_df.min()
```

	0
longitude	-124.3500
latitude	32.5400
housing_median_age	1.0000
total_rooms	2.0000
total_bedrooms	1.0000
population	3.0000
households	1.0000
median_income	0.4999
median_house_value	14999.0000

dtype: float64

```
#maximum value in each column
housing_df.max()
```

	0
longitude	-114.3100
latitude	41.9500
housing_median_age	52.0000
total_rooms	37937.0000
total_bedrooms	6445.0000
population	35682.0000
households	6082.0000
median_income	15.0001
median_house_value	500001.0000

dtype: float64

```
# all the statistical measures about the dataframe
housing_df.describe()
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income
count	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000	17000.000000
mean	-119.562108	35.625225	28.589353	2643.664412	539.410824	1429.573941	501.221941	3.883578
std	2.005166	2.137340	12.586937	2179.947071	421.499452	1147.852959	384.520841	1.908157
min	-124.350000	32.540000	1.000000	2.000000	1.000000	3.000000	1.000000	0.499900
25%	-121.790000	33.930000	18.000000	1462.000000	297.000000	790.000000	282.000000	2.566375
50%	-118.490000	34.250000	29.000000	2127.000000	434.000000	1167.000000	409.000000	3.544600
75%	-118.000000	37.720000	37.000000	3151.250000	648.250000	1721.000000	605.250000	4.767000
max	-114.310000	41.950000	52.000000	37937.000000	6445.000000	35682.000000	6082.000000	15.000100

```
#removing a row
housing_df.drop(index=0,axis=0)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	median_house_value
1	-114.47	34.40	19.0	7650.0	1901.0	1129.0	463.0	1.8200	80100.0
2	-114.56	33.69	17.0	720.0	174.0	333.0	117.0	1.6509	85700.0
3	-114.57	33.64	14.0	1501.0	337.0	515.0	226.0	3.1917	73400.0
4	-114.57	33.57	20.0	1454.0	326.0	624.0	262.0	1.9250	65500.0
5	-114.58	33.63	29.0	1387.0	236.0	671.0	239.0	3.3438	74000.0
...	...	...	...	...	...	...	...	...	...
16995	-124.26	40.58	52.0	2217.0	394.0	907.0	369.0	2.3571	111400.0

```
#removing a column
housing_df.drop(columns='median_house_value',axis=1)
```

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	households	median_income	price
0	-114.31	34.19	15.0	5612.0	1283.0	1015.0	472.0	1.4936	66900.0
1	-114.47	34.40	19.0	7650.0	1901.0	1129.0	463.0	1.8200	80100.0
2	-114.56	33.69	17.0	720.0	174.0	333.0	117.0	1.6509	85700.0
3	-114.57	33.64	14.0	1501.0	337.0	515.0	226.0	3.1917	73400.0
4	-114.57	33.57	20.0	1454.0	326.0	624.0	262.0	1.9250	65500.0

```
#locating a row using the index value
housing_df.iloc[2]
```

	2
longitude	-114.5600
latitude	33.6900
housing_median_age	17.0000
total_rooms	720.0000
total_bedrooms	174.0000
population	333.0000
households	117.0000
median_income	1.6509
median_house_value	85700.0000
price	85700.0000

Data Cleaning:

Data cleaning means fixing bad data in your data set.

Bad data could be:

- Empty cells
- Data in wrong format
- Wrong data
- Duplicates

Example

Remove all rows with NULL values:

```
import pandas as pd
df = pd.read_csv('data.csv')
df.dropna(inplace = True)
print(df.to_string())
```

Duplicates:

Duplicate rows are rows that have been registered more than one time.

```
import pandas as pd
df = pd.read_csv('data.csv')
print(df.duplicated())
```